

Student Details (Student should fill the content)				
Name	Gobiha Mahenthiran			
Student ID	JF/BSCSD/18/44			
Scheduled unit details				
Unit code	CIS6003			
Unit title	Advanced Programming_S1SRI_25			
Unit enrolment details	Year	3		
	Study period			
Lecturer				
Mode of delivery	Full Time			
Assignment Details				
Nature of the Assessment	Course work 100%			
Topic of the Case Study				
Learning Outcomes covered	1,2,3			
Word count				
Due date / Time				
Extension granted?	Yes	No	Extension Date	
Is this a resubmission?	Yes	No	Resubmission Date	
Declaration				
I certify that the attached material is my original work. No other person's work or ideas have been used without acknowledgement. Except where I have clearly stated that I have used some of this material elsewhere, I have not presented it for examination/assessment in any other course or unit at this or any other institution				
Name/Signature			Date	02/27/2026
Submission				
Return to:				

Result				
Marks by 1 st Assessor		Signature of the 1 st Assessor		Agreed Mark
Marks by 2 nd Assessor		Signature of the 2 nd Assessor		

Abstract

The hospitality industry increasingly relies on digital systems to improve operational efficiency and enhance customer experience. Manual reservation systems often cause booking conflicts, inefficient resource allocation, and poor customer management. This project presents the design and development of the Ocean View Resort Reservation Management System, a distributed web-based application developed using Java, JSP, Servlets, and MySQL.

The system enables customers to browse rooms, make reservations, and manage bookings online, while administrators can manage rooms, packages, staff, and clients through an administrative dashboard. The application follows modern software engineering principles including object-oriented design, database management, distributed architecture, and version control practices.

Unified Modeling Language (UML) diagrams such as Use Case Diagrams, Class Diagrams, and Sequence Diagrams were used to model the system design. The application integrates web services and database connectivity using JDBC to ensure efficient data management.

To ensure reliability and quality, the system was tested using structured test planning and automated testing strategies, following principles of Test Driven Development (TDD). Furthermore, the project was managed using Git version control, enabling continuous updates and collaborative development.

The final system successfully demonstrates how modern web technologies can improve hotel reservation management while ensuring scalability, maintainability, and usability.

Table of Contents

1. Introduction.....	5
2. System Overview	7
2.1 Customers	9
2.2 Administrators.....	9
3. Task A – System Design (UML Diagrams).....	10
3.1 Use Case Diagram.....	10
3.2 Class Diagram.....	11
3.3 Sequence Diagram	13
4. Task B – System Development	14
4.1 System Architecture	14
4.2 Database Design.....	14
4.3 Input Validation.....	16
4.4 Reports Generated by the System.....	16
5. Task C – Testing Strategy	17
5.1 Test Plan.....	17
5.2 Test Cases.....	20
5.3 Test Driven Development (TDD)	21
6. Task D – Version Control Using Git	22
6.1 Git Repository	22
6.2 Git Workflow	23
7. Discussion	24
8. Conclusion	26
9. References.....	28

1. Introduction

The hospitality industry relies heavily on efficient reservation management systems to handle guest bookings, maintain room availability, and ensure smooth hotel operations. Traditionally, many hotels managed reservations manually using paper records or spreadsheets. However, such methods often lead to errors, booking conflicts, and delays in service. As hotels grow and customer demand increases, manual systems become inefficient and unreliable.

Ocean View Resort, located in Galle, Sri Lanka, is a popular beachfront hotel that serves hundreds of guests every month. Due to the increasing number of reservations, the hotel management currently faces several operational challenges. The manual reservation process causes booking conflicts, difficulty in tracking guest information, and delays when calculating bills. In addition, manual data storage increases the risk of data loss and reduces overall operational efficiency.

To address these issues, a computerized **Online Room Reservation System** has been proposed. The system will allow hotel staff to manage reservations electronically using a user-friendly interface. The application will store guest information, manage room availability, calculate bills automatically, and provide quick access to reservation details. This system will reduce human errors, improve data management, and enhance customer service quality.

The main objective of this project is to design and develop a reservation management system using **Java programming language, database integration, and modern software development practices**. The system will include authentication, reservation management, billing calculation, and reporting features. In addition, the system design will follow object-oriented principles and implement appropriate design patterns to ensure maintainability and scalability.

This report documents the system design, development process, testing methodology, and version control practices used in the project. It includes UML diagrams such as use case diagrams, class diagrams, and sequence diagrams to illustrate system structure and functionality. Furthermore, the report explains how the application was implemented using a database-driven architecture and how automated testing was performed to ensure system reliability.

The Ocean View Resort Reservation Management System was developed using **Java Web Technologies**, including Java Servlets and JSP (Java Server Pages) for building dynamic web interfaces. The system communicates with a **MySQL database using JDBC (Java Database Connectivity)** to store and retrieve information related to users, rooms, reservations, and packages. The architecture of the system follows the **Model-View-Controller (MVC)** design pattern, which separates the application into three main components: the model, view, and controller. This design approach improves system organization by separating business logic from the user interface and database operations. As a result, the application becomes easier to maintain, update, and extend in the future.

Additionally, the system incorporates **input validation mechanisms** to ensure that users provide correct and complete information when interacting with the application. Validation prevents invalid data entries such as empty fields, incorrect date selections, or unauthorized access attempts. The system also includes reporting capabilities that allow administrators to analyze reservation records and operational statistics. These reports provide valuable insights that help hotel management improve service quality and optimize resource allocation.

Overall, the Ocean View Resort Reservation Management System demonstrates how modern web technologies can be applied to solve common challenges in hotel management. By integrating distributed web services, database management, and structured software design principles, the system provides a reliable and scalable solution that enhances both customer experience and administrative efficiency.

2. System Overview

The **Ocean View Resort Reservation System** has been designed to efficiently manage the hotel's reservation process by supporting two primary user roles: **customers** and **administrators**. Each role interacts with the system in a different way, depending on their responsibilities and access permissions. By clearly separating these roles, the system ensures both usability for customers and effective management capabilities for administrators.

Customers represent the external users of the system who wish to book accommodation at the Ocean View Resort. The system allows customers to **register and create a personal account**, which enables them to securely access the booking platform. During registration, users provide essential information such as their name, contact details, and login credentials. Once registered, customers can log into the system and explore the available services provided by the resort. One of the most important features available to customers is the ability to **view available rooms** along with their details, such as room type, price, and availability status. This allows customers to make informed decisions before proceeding with a reservation.

After reviewing the available rooms, customers can **make reservations directly through the system**. The booking process allows users to select one or multiple rooms and specify their desired check-in and check-out dates. The system then verifies the availability of the selected rooms and calculates the total cost based on the duration of the stay. Once confirmed, the reservation is stored in the system's database and the room status is updated to prevent double booking. Customers can also **view their booking details**, allowing them to track their reservations and confirm their stay information whenever necessary. This functionality improves convenience and transparency for customers.

In contrast, administrators are responsible for managing the internal operations of the hotel. The system provides administrators with a **dashboard that offers complete control over system data and hotel resources**. Through this interface, administrators can manage hotel rooms by adding new rooms, updating existing room information, or removing rooms that are no longer available. This functionality ensures that the system always reflects the actual room availability within the hotel.

Administrators also have the ability to **manage hotel packages**, which may include special offers, pricing plans, or seasonal promotions. By updating package information in the system, administrators can adjust pricing and services according to business requirements. In addition to managing rooms and packages, administrators can also **manage staff and client accounts**. This includes creating new staff profiles, updating user information, and removing inactive accounts. These features help maintain accurate records and ensure proper system access control.

Another important responsibility of administrators is to **monitor reservations and generate reports**. By reviewing reservation records, administrators can track hotel occupancy, analyze booking trends, and evaluate overall system usage. These reports provide valuable insights that support better decision-making and operational planning.

The architecture of the Ocean View Resort Reservation System is structured into several layers to ensure efficiency, maintainability, and scalability. The **Presentation Layer** is responsible for displaying the user interface through JSP pages and HTML components, allowing users to interact with the system. The **Application Layer** contains Java Servlets and controllers that handle user requests and business logic. The **Data Access Layer** uses DAO classes with JDBC to manage communication between the application and the database. Finally, the **Database Layer** stores all essential information in a MySQL database, including user data, room details, packages, and reservations. This layered architecture promotes modular development and ensures that each component of the system performs a specific role within the overall application structure.

The system architecture consists of the following layers:

Layer	Description
Presentation Layer	JSP pages and HTML interface
Application Layer	Java Servlets and Controllers
Data Access Layer	DAO classes using JDBC
Database Layer	MySQL database storing system data

2.1 Customers

Customers can:

- Register and log into the system
- View available rooms
- Make reservations
- View booking details

2.2 Administrators

Administrators can:

- Manage hotel rooms
- Manage packages
- Manage staff and clients
- View reservations
- Generate reports

3. Task A – System Design (UML Diagrams)

Unified Modeling Language (UML) diagrams were used to visualize and design the structure and behavior of the Ocean View Resort Reservation Management System. UML provides a standardized way to represent the system's components, relationships, and interactions before the actual development begins. By using UML diagrams, developers can better understand how different parts of the system work together and how users interact with the application. In this project, several UML diagrams such as the Use Case Diagram, Class Diagram, and Sequence Diagram were created to represent the system architecture. These diagrams helped identify system functionalities, define relationships between classes, and illustrate the flow of operations during the reservation process.

3.1 Use Case Diagram

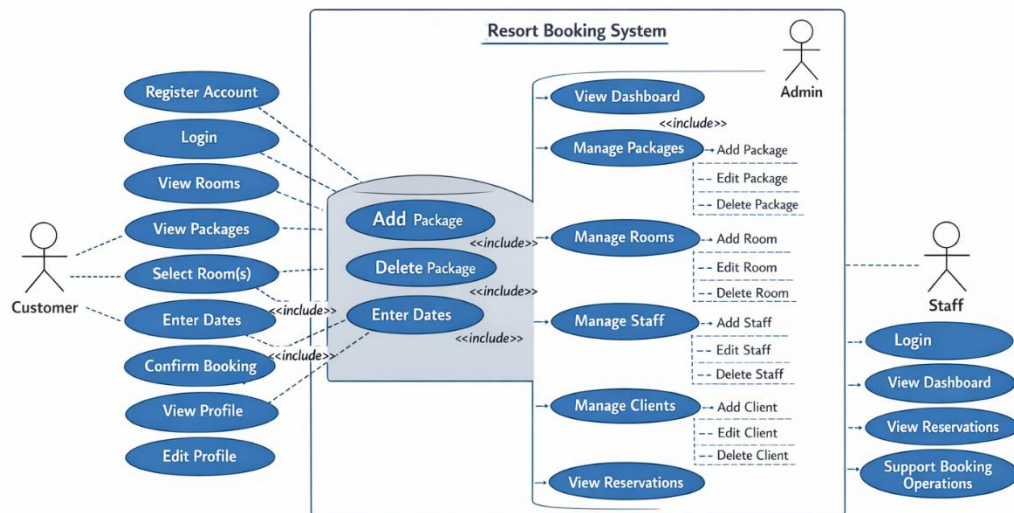
The Use Case Diagram identifies the interactions between users and the system.

Main Actors:

- Admin
- Customer

Main Use Cases:

- Login
- Register
- View Rooms
- Make Reservation
- Manage Rooms
- Manage Packages
- Manage Staff
- View Reservations



The Use Case Diagram illustrates the relationship between system actors and the functionalities provided by the system. Customers interact with the system to view available rooms and create reservations, while administrators manage operational components such as rooms, packages, staff members, and clients. This design helps clearly define system responsibilities and user interactions.

3.2 Class Diagram

The Class Diagram represents the structure of the system including classes, attributes, and relationships.

Key Classes:

Users

Rooms

Packages

Reservations

RoomDAO

ReservationDAO

UserDAO

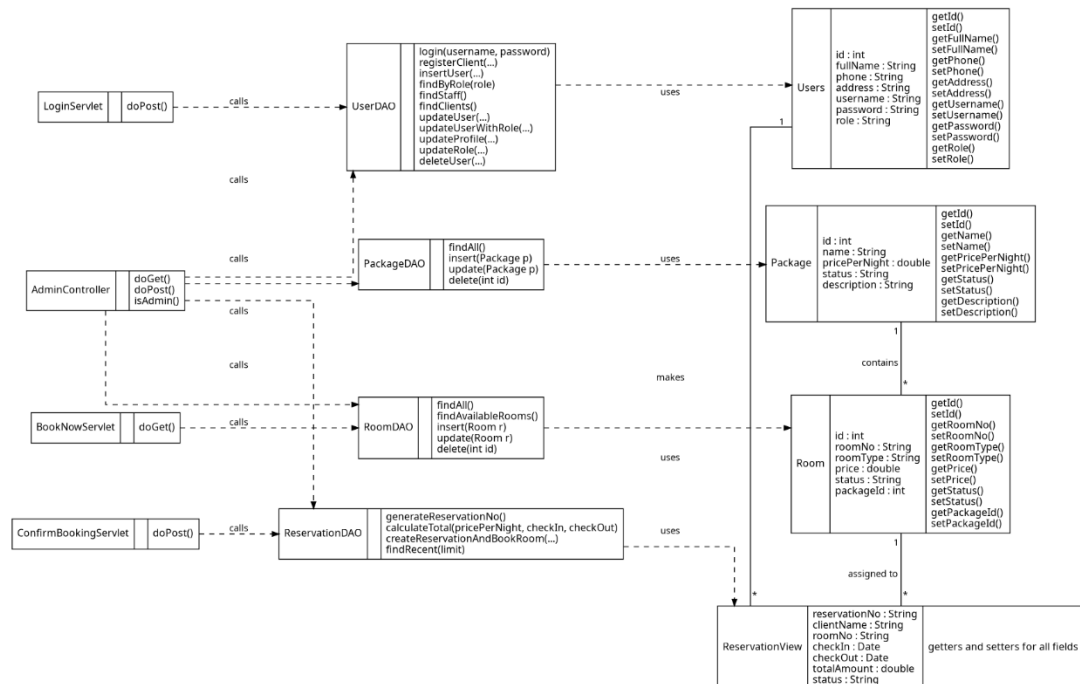
AdminController

Example:

The class diagram represents the static structure of the system. Each entity corresponds to a real-world object in the hotel environment. For example, the Room class represents

hotel rooms with attributes such as room number, type, price, and availability status. The Reservation class stores booking details including reservation number, check-in date, check-out date, and total amount.

The DAO (Data Access Object) classes are responsible for interacting with the database using JDBC. This design pattern separates business logic from database operations, improving maintainability and scalability.



3.3 Sequence Diagram

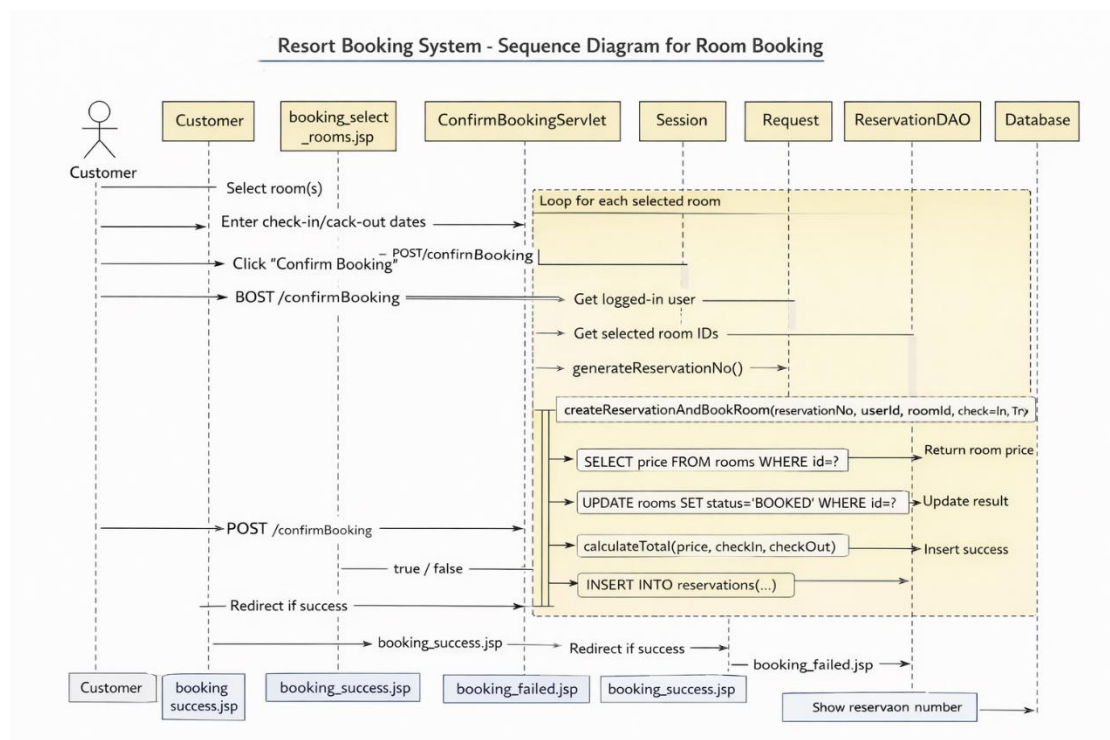
The Sequence Diagram illustrates the process of making a reservation.

Steps:

1. User selects room
2. System checks room availability
3. System processes reservation
4. Reservation stored in database
5. Confirmation returned to user

Example:

The sequence diagram demonstrates the interaction between system components during the reservation process. When a user selects a room and confirms a booking, the request is sent to the controller. The controller communicates with the DAO layer to verify room availability and store reservation details in the database. Finally, the system returns a confirmation message to the user.



4. Task B – System Development

4.1 System Architecture

The system follows the **Model View Controller (MVC)** architecture.

Model

Database Entities and DAO classes

View

JSP pages (user interface)

Controller

Java Servlets handling requests

This architecture separates application logic from user interface components, making the system easier to maintain and extend.

4.2 Database Design

The system uses MySQL database with the following tables:

Users

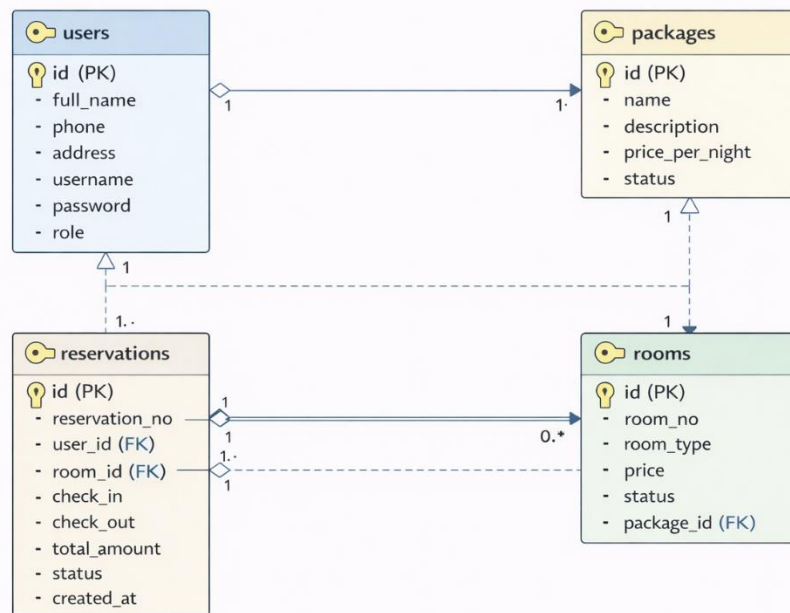
Rooms

Packages

Reservations

Example:

The database stores essential information related to users, rooms, packages, and reservations. The reservations table links users and rooms through foreign key relationships.



4.3 Input Validation

Input validation is implemented in the system to ensure that only accurate and valid information is entered into the database. This helps prevent errors and improves the reliability of the reservation system. Several validation methods are applied, including **required field validation** to ensure that important fields such as name, username, and password are not left empty. **Date validation** ensures that check-in and check-out dates are correctly selected and logically valid. **Role selection validation** confirms that a valid user role is chosen during account creation. Additionally, **login authentication** verifies user credentials before granting system access. Both **client-side and server-side validation** are used to maintain data integrity and system security.

4.4 Reports Generated by the System

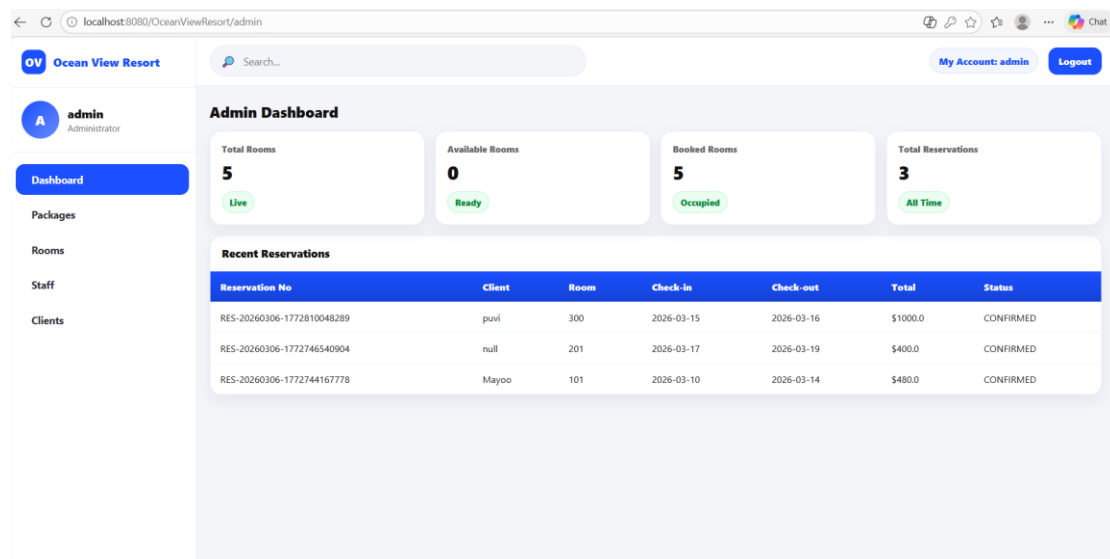
The system provides several useful reports:

Total Rooms

Available Rooms

Booked Rooms

Total Reservations



The dashboard provides administrators with a quick overview of system statistics and recent reservations.

5. Task C – Testing Strategy

System Testing and Quality Assurance

Testing is an essential phase of the software development lifecycle. It ensures that the developed system functions correctly, meets user requirements, and performs reliably under different conditions. In the Ocean View Resort Reservation Management System, testing was carried out to verify that all system components work together correctly and that the application behaves as expected when users interact with it.

The main objective of testing in this project was to detect errors early, validate system functionality, and ensure that the system provides a smooth and reliable experience for both administrators and customers. The testing process also helped verify that database operations, user inputs, and reservation processes function correctly without producing unexpected results.

A structured testing strategy was followed in this project, including **Unit Testing, Integration Testing, System Testing, and User Acceptance Testing**. Each testing stage focused on different aspects of the system to ensure overall quality and stability.

5.1 Test Plan

The test plan defines the strategy used to verify the correctness and reliability of the system. It describes the testing objectives, testing types, testing environment, and testing procedures used to validate the application.

For the Ocean View Resort Reservation System, the test plan focused on verifying key functionalities such as user login, room management, reservation creation, and database interactions. Since the system is a web-based distributed application developed using **Java Servlets, JSP, JDBC, and MySQL**, testing was conducted to ensure that the front-end interface correctly communicates with the back-end database.

The testing environment included the following components:

- **Development Environment:** NetBeans IDE
- **Web Server:** Apache Tomcat
- **Database System:** MySQL
- **Browser Testing:** Google Chrome
- **Operating System:** Windows

Several testing techniques were applied to evaluate different layers of the system. These testing methods ensured that both individual components and the complete system function correctly.

Unit Testing

Unit testing focuses on verifying individual components of the system independently. In this project, unit testing was performed on DAO classes and controller methods to ensure that database operations such as inserting reservations, updating room status, and retrieving data function correctly.

For example, individual methods in the **ReservationDAO**, **UserDAO**, and **RoomDAO** classes were tested to verify that they correctly interact with the database. Methods responsible for inserting reservations and updating room availability were tested with different inputs to confirm that they behave as expected.

Integration Testing

Integration testing ensures that multiple components of the system work correctly when combined. In this stage, the interaction between JSP pages, Servlets, DAO classes, and the database was tested.

For example, when a user selects rooms and confirms a booking, the request passes through several layers of the system. The JSP page collects user inputs, the servlet processes the request, and the DAO class updates the database. Integration testing verified that these components work together without errors.

System Testing

System testing evaluates the complete application to ensure that the entire system functions as intended. During this stage, the full reservation process was tested from start to finish. This included user registration, login authentication, viewing available rooms, selecting rooms, confirming reservations, and viewing booking results.

System testing also verified that role-based access control works properly. For example, administrators should be able to access the dashboard and manage system data, while customers should only be able to make reservations and view their bookings.

User Acceptance Testing

User Acceptance Testing (UAT) ensures that the system satisfies the needs of end users. In this stage, the application was tested from the perspective of real users to confirm that it is easy to use and performs expected tasks correctly.

Users tested important features such as booking rooms, viewing packages, and navigating the system interface. Feedback from this stage helped identify minor usability improvements and confirm that the system meets the functional requirements specified at the beginning of the project.

5.2 Test Cases

Test cases were created to validate important functionalities of the reservation system. Each test case included input values, expected outputs, and the actual result observed during testing.

Below are some key test cases used in the system.

Login Test

This test verifies whether the system correctly authenticates users when valid login credentials are provided.

Test Case	Input	Expected Output
Login Test	Correct username and password	User successfully logs into the system and is redirected to the appropriate dashboard

During testing, valid administrator and client credentials were entered into the login form. The system successfully authenticated the user and redirected them to the correct interface depending on their role.

Booking Test

This test checks whether the system can successfully create a reservation when valid room selections and dates are provided.

Test Case	Input	Expected Output
Booking Test	Valid room selection and valid check-in/check-out dates	Reservation successfully created and stored in the database

The test confirmed that the reservation information was correctly saved in the **reservations table**, and the room status was updated to **BOOKED**.

Validation Test

This test verifies that the system prevents invalid inputs from being submitted.

Test Case	Input	Expected Output
Validation Test	Empty required fields	System displays an error message and prevents submission

For example, if a user attempts to submit the booking form without selecting any rooms or entering dates, the system displays a validation error message. This ensures that incomplete or incorrect data cannot enter the database.

These test cases demonstrate that the system correctly handles both valid and invalid inputs while maintaining data integrity.

5.3 Test Driven Development (TDD)

Test Driven Development (TDD) is a software development approach where test cases are written before implementing the actual functionality. The main objective of TDD is to improve code quality, detect errors early, and ensure that each feature meets its intended requirements.

In this project, the principles of Test Driven Development were applied during the implementation of several system features, particularly in the reservation and user authentication modules.

The TDD process followed four main steps.

Step 1 – Write Test Cases

Before implementing new functionality, test cases were designed to define the expected behavior of the system. For example, before implementing the reservation creation method, test scenarios were created to verify whether the system correctly handles room booking, date validation, and database storage.

Step 2 – Implement Functionality

After defining the test cases, the required functionality was implemented in the system. For example, the `createReservationAndBookRoom()` method was developed in the `ReservationDAO` class to insert reservation records and update room status.

Step 3 – Run Tests

Once the functionality was implemented, the test cases were executed to verify that the new feature works as expected. If a test failed, the code was reviewed and corrected until the functionality met the expected outcome.

Step 4 – Refactor Code

After the tests passed successfully, the code was improved to enhance readability, performance, and maintainability. Refactoring ensured that the code remained clean and well-structured without affecting the system's behavior.

Applying the TDD approach helped reduce the number of software defects and improved overall code reliability. It also ensured that each feature of the system was properly tested before integration into the main application.

Summary

Testing played a crucial role in ensuring the reliability and performance of the Ocean View Resort Reservation Management System. By applying multiple testing strategies, including unit testing, integration testing, system testing, and user acceptance testing, the system was verified to function correctly under various conditions.

Additionally, the use of Test Driven Development helped improve software quality by ensuring that new features were tested before being fully implemented. This structured testing approach contributed to the development of a stable, secure, and efficient reservation management system.

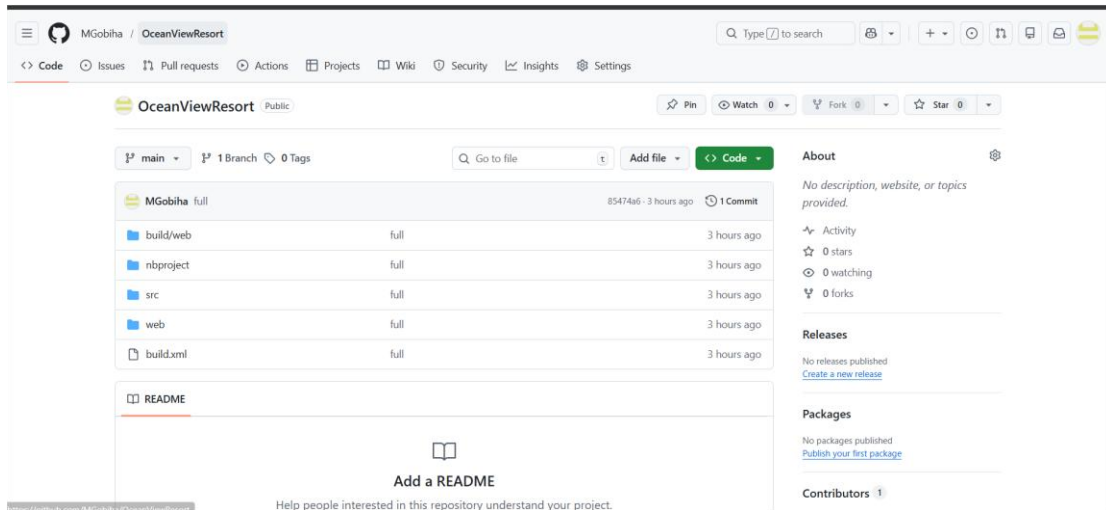
6. Task D – Version Control Using Git

6.1 Git Repository

The project was managed using Git version control and hosted on GitHub.

Repository link:

<https://github.com/MGobiha/OceanViewResort.git>



6.2 Git Workflow

The following workflow was followed:

1. Create repository
2. Commit initial project files
3. Add features gradually
4. Push updates daily
5. Track project versions

7. Discussion

The Ocean View Resort Reservation Management System demonstrates how modern web technologies can be used to improve and automate traditional hotel management processes. In many hotels, reservations and customer information are still managed manually or through outdated systems, which can lead to problems such as booking conflicts, data entry mistakes, and inefficient handling of customer records. By implementing a web-based reservation system, these issues can be significantly reduced while improving the overall operational efficiency of the hotel.

One of the key benefits of this system is the ability to manage reservations in real time. Customers are able to view available rooms, select suitable dates, and confirm bookings directly through the online platform. This eliminates the need for manual reservation handling and reduces the risk of double bookings. At the same time, administrators are able to monitor and manage the hotel's operations through a centralized dashboard. They can easily update room details, manage packages, control staff accounts, and review reservation records. This centralized management approach makes it easier to organize hotel operations and maintain accurate records.

Another major strength of the system is the use of the **Model–View–Controller (MVC) architectural pattern**. MVC divides the system into three main components: the model, view, and controller. The model handles the data and business logic of the application, the view represents the user interface through JSP pages, and the controller manages user requests through Java servlets. This separation of responsibilities helps keep the code organized and easier to maintain. For example, if the user interface needs to be updated, developers can modify the view layer without affecting the business logic or database structure.

In addition to MVC, the system also implements the **Data Access Object (DAO) design pattern**. DAO classes handle communication between the application and the database using JDBC. By isolating database operations inside DAO classes, the system becomes easier to manage and maintain. Any future changes to the database structure can be handled within the DAO layer without affecting other parts of the application. This improves the scalability and flexibility of the system.

Another important aspect of the project is the implementation of **data validation and testing mechanisms**. Input validation ensures that incorrect or incomplete information

cannot be submitted by users. For example, the system checks that required fields are filled, booking dates are valid, and user roles are properly selected. These validation processes help protect the database from invalid entries and maintain system reliability.

Testing also played a critical role in ensuring system quality. Various testing approaches such as unit testing, integration testing, and system testing were applied to confirm that each component functions correctly. These tests verified important system operations including login authentication, room booking, and reservation storage in the database.

Despite the successful implementation of the current features, there are several opportunities for future improvements. One possible enhancement is the development of a **mobile-compatible version** of the system so that users can easily access the reservation platform using smartphones and tablets. Another valuable improvement would be the integration of **online payment gateways**, allowing customers to complete payments directly when making reservations. This would make the booking process more convenient and efficient.

Security improvements could also be considered for future development. For instance, implementing **password encryption, secure authentication methods, and user activity monitoring** would further strengthen the system's protection against unauthorized access.

Overall, the system provides a strong foundation for a modern hotel reservation platform while leaving room for future enhancements.

8. Conclusion

The Ocean View Resort Reservation Management System was successfully designed and implemented as a distributed web-based application using Java technologies. The main goal of this project was to develop an efficient system capable of managing hotel reservations, customer information, and administrative operations in a structured and automated manner.

Throughout the development process, several modern software engineering practices were applied. The system architecture was carefully designed using **UML diagrams**, including use case diagrams, class diagrams, and sequence diagrams. These diagrams helped visualize the system structure and interactions between different components before the actual implementation began. Proper planning through UML modeling contributed to the creation of a well-structured and maintainable system.

The project also applied the **Model–View–Controller (MVC) architecture**, which separates the system into different layers responsible for handling data, user interface, and request processing. This architecture improves the maintainability and scalability of the system by ensuring that each component has a clearly defined role. In addition, the use of **DAO design patterns** simplified database communication and improved code organization.

Database integration was achieved using **MySQL and JDBC**, which allowed the system to store and manage large amounts of information such as user accounts, room details, packages, and reservations. The database design ensured proper relationships between tables, allowing the system to retrieve and manage data efficiently.

Testing strategies were also implemented to verify that the system works correctly under different conditions. Several test cases were created to evaluate critical functionalities such as login authentication, reservation creation, and input validation. These testing procedures helped ensure that the system behaves as expected and that potential errors were identified and corrected during development.

Version control techniques were used through a **GitHub repository**, allowing the project to track development progress and maintain multiple versions of the software. By uploading regular updates and documenting changes, the development process became more organized and transparent.

From a user perspective, the system provides a **simple and user-friendly interface** for customers to view available rooms and make reservations. At the same time, administrators are given powerful management tools to control different aspects of the hotel system through an administrative dashboard.

Although the system already fulfills its primary objectives, future improvements could further enhance its capabilities. Integrating secure **online payment systems**, developing **mobile applications**, and implementing **advanced data analytics for hotel management** could significantly increase the value of the platform.

In conclusion, the Ocean View Resort Reservation Management System successfully demonstrates how modern web technologies and software engineering principles can be applied to develop a practical and efficient reservation management solution. The system not only automates essential hotel operations but also provides a scalable foundation for future development and innovation in hospitality management systems.

9.References

Anon., Last Updated : 9 Sep, 2025. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/java-design-patterns/>

Anon., n.d. *MySQL (2024) MySQL Documentation*. [Online]

Available at: <https://dev.mysql.com/doc/>

Anon., n.d. *Oracle (2024) Java Servlet Technology*. [Online]

Available at: <https://docs.oracle.com/en/>

Anon., n.d. *w3schools.com*. [Online]

Available at: <https://www.w3schools.com/css/>

Kumar, D. A., 6 March 2022. *jcodebook.com*. [Online]

Available at: <https://jcodebook.com/how-to-connect-servlet-program-with-mysql-database/>